

北京邮电大学

《编译原理与技术课程设计》报告

指导教师：王吴凡

姓名	班级	学号	备注
郝政伟	304	2023210979	组长
周柄名	304	2023210992	
李良顺	304	2023210984	
蔡禹煊	304	2023210985	
张睿渤	304	2023210991	
吴锡华	304	2023210998	
孔力帆	304	2023210999	

计算机学院（国家示范性软件学院）

2026 年 5 月

目 录

1. 课程设计任务和目标	3
2. 需求分析	4
2.1 源语言范围与输入输出形式	4
2.2 功能需求分析	4
2.3 错误处理与可用性需求	4
2.4 开发环境与验证环境	5
3. 总体设计	5
3.1 设计目标与总体思路	5
3.2 核心数据结构设计	5
3.3 总体结构设计	6
3.4 模块划分与模块关系	6
3.5 模块接口设计	7
3.6 用户接口设计	7
4. 详细设计	7
4.1 词法分析模块详细设计	7
4.2 语法分析与 AST 构建详细设计	8
4.3 语义分析、符号表与类型系统详细设计	8
4.4 代码生成模块详细设计	8
4.5 错误处理与测试支撑详细设计	9
5. 源程序清单	10
5.1 源程序组织方式	10
5.2 核心源文件清单	10
5.3 测试与辅助文件清单	11
5.4 代码组织与实现特点	11
5.5 源程序提交说明	12
6. 程序测试	12
6.1 测试环境	12
6.2 测试内容与测试分类	12
6.3 代表性测试用例	13
6.4 测试结果	13
6.5 预期结果与实际结果对比	14
6.6 测试结果分析	14
7. 课程设计总结	14
7.1 主要完成工作	14
7.2 设计与实现过程中的主要问题	15
7.3 分工协作与个人收获	15
7.4 不足与后续改进方向	15
7.5 总结	16

1. 课程设计任务和目标

《编译原理与技术课程设计》的核心任务是设计并实现一个 Pascal-S 语言编译程序，使其能够对 Pascal-S 源程序完成词法分析、语法分析、语义分析和目标代码生成，并将输入程序翻译为等价的 C 语言程序。该项目是编译原理课程的综合实践环节，要求将课堂中涉及的形式语言、有限自动机、语法分析、符号表管理、语义检查与代码生成等理论落实到一个可运行、可验证的系统软件中。

本项目的目标不仅是“实现一个能跑的编译器”，还包括以下三个层面的要求。第一，在功能层面，编译器应能够正确处理 Pascal-S 子集中的程序结构、声明、表达式、控制流、函数过程调用、数组访问等基本结构，并在此基础上支持扩展功能。第二，在工程层面，系统应具备清晰的模块划分、稳定的构建方式、可追踪的错误处理机制以及可复现的测试方法。第三，在课程设计验收层面，项目应形成较完整的文档、测试和使用说明，并能够在平台测试、现场演示和答辩问答中自洽地说明设计思路与实现过程。

围绕上述目标，本组最终采用了“词法分析 -> 语法分析与 AST 构建 -> 语义分析 -> C 代码生成”的流水线结构，并在工程实现中引入独立的错误处理模块、共享的类型解析模块以及课程自建测试集。与较早期版本相比，系统不再把符号表维护和类型判断混入 parser 中，而是通过独立的语义分析阶段完成标识符绑定、作用域管理和类型检查，从而使整体架构更接近经典编译器设计。

截至当前验收准备阶段，系统已经能够稳定完成 Pascal-S 到 C 的翻译，支持变量、常量、函数、过程、数组、布尔字面量、控制流语句、函数返回值处理、record 结构与多级字段访问，以及 repeat ... until 语句扩展；同时支持带源码定位的 caret diagnostics，并建立了课程自建测试体系。在头歌平台评测方面，本项目已经达到 95 / 95 全部通过的结果，本地还补充了成功样例、错误样例和严格生成代码检查，用于支撑最终答辩和书面报告。

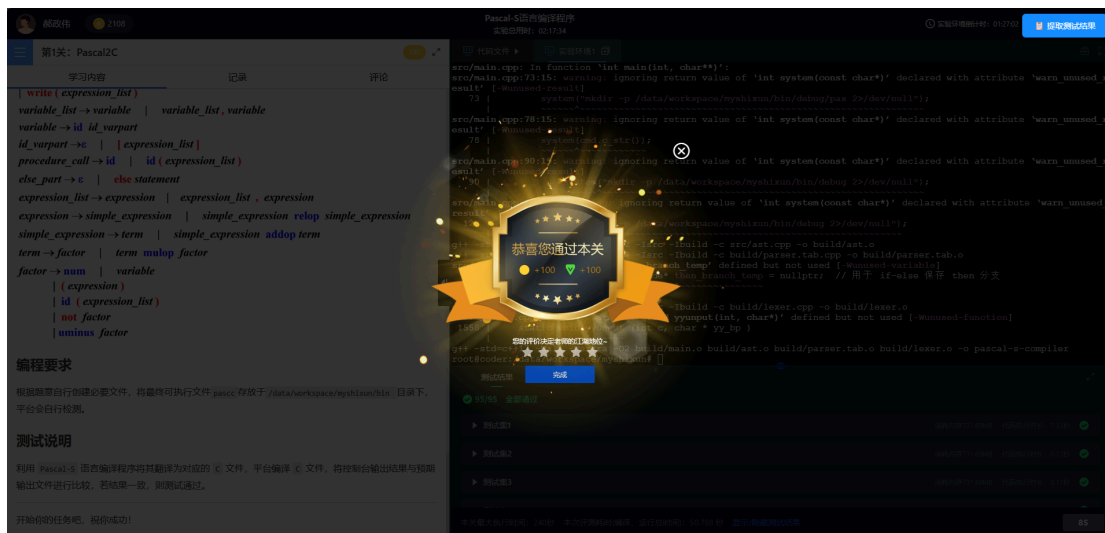


Figure 1: 头歌平台最终评测结果：95 / 95 全部通过

2. 需求分析

2.1 源语言范围与输入输出形式

本系统面向的源语言是课程给定的 Pascal-S 语言子集。输入为符合 Pascal-S 语法的 `.pas` 源文件，输出为等价的 `.c` 目标文件，后续可由 GCC 等 C 编译器进一步编译并执行。与传统“直接生成汇编”的课程设计路线相比，本项目采用 Pascal-S 到 C 的源到源翻译方式，使得目标代码更易于调试、验证和展示，同时能够借助成熟的 C 编译环境快速完成运行结果验证。

从语言能力范围来看，系统需要支持 Pascal-S 中的程序头、常量声明、变量声明、函数与过程定义、复合语句、赋值语句、条件语句、循环语句、数组访问、函数/过程调用以及输入输出语句等基本结构。本项目在完成基本要求的同时，还进一步补充了 `record` 结构与多级字段访问支持，使语言子集更加完整，也为课程设计加分项提供了可展示的扩展能力。

2.2 功能需求分析

结合课程要求和平台评测标准，系统的功能需求可分为四类。

第一类是词法分析功能。系统需要能够识别 Pascal-S 中的关键字、标识符、整数字面量、实数字面量、布尔字面量、字符/字符串字面量、运算符和界符，并过滤空白符和注释。由于 Pascal-S 对关键字与标识符大小写不敏感，因此词法分析阶段需要对标识符进行统一化处理。同时，词法分析还需要记录每个 token 的行号和列号，为后续错误定位提供基础数据。

第二类是语法分析功能。系统需要根据 Pascal-S 的文法规则对 token 流进行归约，识别程序头、声明部分、函数和过程、表达式、语句块以及控制流结构，并在归约过程中构建 AST。语法分析阶段的核心任务不是简单判断“句子是否合法”，而是把源程序组织成后续阶段可统一处理的中间结构。

第三类是语义分析功能。语法正确并不意味着程序在含义上正确，因此系统还需要建立符号表并进行上下文相关检查，包括：标识符是否声明、同一作用域内是否重复声明、赋值语句左右类型是否兼容、函数与过程调用的参数个数和类型是否匹配、引用参数是否为左值、条件表达式是否满足布尔兼容要求，以及 `record` 字段访问是否合法等。考虑到 Pascal-S 中函数返回值通常通过“对函数名赋值”实现，本项目在语义与后端阶段还需对这一语义进行正确处理。

第四类是代码生成功能。系统需要把 AST 中的声明、表达式、控制流和子程序结构翻译成等价的 C 代码，并处理 Pascal-S 与 C 在语义上的差异。例如 Pascal-S 数组支持任意下界，而 C 数组默认从 0 开始，因此生成数组访问时必须加入偏移计算；`var` 参数在 Pascal-S 中对应引用传递，在 C 中需要映射为指针参数；`record` 则需要映射为 C 的 `struct` 结构。最终输出的 C 代码必须可编译、可运行，且运行结果与源程序语义保持一致。

2.3 错误处理与可用性需求

课程设计对编译器的健壮性提出了明显要求，因此系统不能只在“合法输入”上工作，还必须能够对错误输入进行诊断并在合适场景下继续处理。本项目在需求层面要求系统支持词法错误、语法错误和语义错误的分类报告，并尽量输出清晰的错误位置和原因说明。为此，系统在词法阶段记录 token 的行列坐标，在主程序中缓存源文件内容，在错误处理模块中统一输出错误类型、源码原文和定位箭头，从而形成更接近现代编译器的 `caret diagnostics` 效果。

除了错误定位外，系统还需要具有基本的可测试性和可交付性。这意味着编译器应具备稳定的命令行入口、可重复执行的构建方式、明确的输入输出规则，以及一组能够覆盖主要功能和错

误场景的测试用例。由于课程验收不仅看代码，还看测试、说明与答辩展示，因此“可运行、可验证、可解释”也是需求分析中的重要组成部分。

2.4 开发环境与验证环境

本项目主要在 Windows 环境下开发，使用的核心工具包括 C++、Flex、Bison、GCC / MinGW、CMake 和 PowerShell 脚本。词法分析器和语法分析器分别由 Flex 与 Bison 自动生成，核心编译逻辑由 C++ 实现，目标代码使用 GCC 验证可编译性与运行结果。在线评测方面，项目最终需要满足头歌平台的公开与隐藏测试要求；本组在本地还补充了 `user_tests` 与 `course_tests` 两类测试材料，用于验证基础功能、扩展功能和错误处理能力。

综合来看，本项目的需求并非单一地追求“通过平台测试”，而是要求实现一个能够把 Pascal-S 源程序稳定翻译为 C 程序、结构清晰、支持错误诊断并具备自建测试支撑的课程设计原型系统。这一需求分析也决定了后续总体设计必须围绕“流水线结构、共享类型信息、统一错误处理和可验证输出”四个关键词展开。

3. 总体设计

3.1 设计目标与总体思路

结合上一章需求分析，本项目在总体设计阶段遵循两个原则。第一，尽量采用清晰的编译流水线结构，把“词法分析、语法分析、语义分析、代码生成”四类职责分离，避免把语义判断和目标代码输出混杂在同一阶段。第二，围绕课程验收的需求，把系统设计成“可运行、可验证、可定位错误”的工程原型，而不只是一个只能通过少量样例的实验程序。

因此，本项目最终没有采用“在 parser 中一边归约一边完成全部语义与输出”的粗放式结构，而是采用了基于 AST 的多阶段流水线。源程序首先经由词法分析和语法分析转换为抽象语法树；然后由独立的语义分析器完成符号表构建、作用域管理和类型检查；最后由代码生成器把合法的 AST 翻译为等价的 C 程序。错误处理模块贯穿各阶段，统一负责错误记录和输出；测试与运行脚本作为外围支撑，保证该系统可以稳定构建和复现。

3.2 核心数据结构设计

总体设计中最重要数据结构是 AST、符号表和类型辅助结构。

首先，AST 作为整个系统的中间表示，承载了 Pascal-S 程序的结构信息。项目中定义了 `ASTNode` 抽象基类，并进一步划分为表达式节点、语句节点、声明节点和程序根节点。表达式节点包括整型/实型/布尔/字符字面量、标识符、数组访问、record 字段访问、二元运算、一元运算和函数调用等；语句节点包括赋值语句、复合语句、条件语句、循环语句、过程调用与输出语句；声明节点包括变量声明和函数/过程声明。这样设计的好处是：语法分析只需要专注于建树，而语义分析和代码生成都可以围绕同一棵树工作。

其次，符号表用于维护上下文相关的信息。项目中的 `SymbolEntry` 保存标识符名称、种类、数据类型、作用域层级、是否为常量、是否为引用参数、数组信息、record 信息、参数列表和返回类型等属性；`SymbolTable` 本身采用作用域栈结构，支持进入作用域、退出作用域、插入、按就近原则查找以及在当前作用域判重。由于 Pascal-S 中函数、过程、变量、数组、record 字段访问都需要依赖上下文解释，因此符号表是语义分析阶段的核心支撑。

第三类是类型辅助结构。为了支持数组与 record，本项目定义了 `ArrayInfo`、`ArrayDimension`、`RecordInfo`、`RecordField` 与 `ParameterInfo` 等结构，用来描述数组维度、上下

界、元素类型、record 字段及参数传递方式。与此同时，考虑到语义分析和代码生成都需要查询表达式类型，本项目引入了独立的 `TypeResolver` 模块，统一提供表达式类型推断、数组元素类型解析和 record 字段类型解析，避免类型规则在多个阶段重复实现。

3.3 总体结构设计

从整体流程上看，本系统的主控入口位于 `main.cpp`。主程序首先解析命令行参数，确定输入输出路径；随后读取源文件各行内容，缓存到错误处理模块中，以支持后续源码级定位；然后打开源文件并调用由 Bison 生成的 `yyparse()`，在解析成功后得到程序的根节点 `ProgramNode`。如果词法或语法阶段已经报告错误，则停止后续流程并统一输出。

在 AST 构建完成后，系统进入语义分析阶段。`SemanticAnalyzer` 作为 `ASTVisitor` 的一个具体实现，遍历整棵 AST，建立和维护独立的符号表，检查变量声明与使用、作用域、赋值类型、函数过程调用、参数类型、引用参数左值约束以及 record 字段访问等问题。语义分析通过后，主程序再调用 `CodeGenerator::generate()` 生成 C 代码，并将结果写出到目标文件。

这一结构的关键点在于：parser 不再直接承担全部编译职责，而是只负责生成结构化 AST；语义分析负责“程序是否成立”；代码生成负责“程序如何映射到目标语言”。相较于早期将符号表副作用和部分类型判断混入 parser 的实现方式，这种设计显著改善了模块边界，使系统更容易扩展、调试和答辩说明。

3.4 模块划分与模块关系

根据当前代码结构，系统可划分为以下几个核心模块。

(1) 词法分析模块。该模块由 `lexer.l` 描述，由 Flex 自动生成扫描器。其职责是从字符流中识别关键字、标识符、字面量、运算符和界符，过滤空白符与注释，并维护行号、列号等位置信息。

(2) 语法分析与解析状态模块。该模块由 `parser.y` 和 `parser_state.*` 组成，由 Bison 生成语法分析器。它根据 Pascal-S 文法对 token 流进行归约，并在归约动作中构建 AST 节点。`ParserState` 用于集中保存解析期的暂存状态，如参数列表、局部变量声明、数组和 record 的附带信息等，从而减少分散的全局可变状态。

(3) AST 与公共数据结构模块。该模块由 `ast.*` 提供统一的 AST 节点体系及访问者接口，既是前端分析的产物，也是中后端处理的共享输入。

(4) 语义分析、符号表与类型系统模块。该模块由 `semantic_analyzer.*`、`symbol_table.*`、`type_resolver.*` 组成。它负责标识符绑定、作用域管理、类型兼容性检查、record 字段合法性检查以及表达式类型解析，是系统保证静态语义正确性的关键阶段。

(5) 代码生成模块。该模块由 `codegen.*` 组成，负责把 AST 翻译成 C 程序，并处理 Pascal-S 与 C 的语义差异，例如数组下界偏移、record 到 `struct` 的映射、引用参数到指针参数的映射、函数返回值变量 `_retval` 的引入等。

(6) 错误处理模块。该模块由 `error.*` 构成，负责收集词法、语法、语义和代码生成阶段的错误，并统一输出带源码原文和定位箭头的诊断信息。

(7) 测试与运行支撑模块。该部分由 `course_tests/`、`user_tests/`、构建脚本和运行脚本组成，负责课程自建测试、错误样例、严格生成代码检查以及平台构建/运行支撑。

模块之间的关系可以概括为：词法分析为语法分析提供 token 流；语法分析构建 AST；语义分析与代码生成都基于 AST 工作，并通过符号表与类型系统查询必要信息；错误处理模块为所有阶段提供统一出口；测试与运行支撑模块验证整个流水线的正确性与可交付性。

3.5 模块接口设计

从接口层面看，主控程序与各模块之间的交互关系比较清晰。词法分析器通过 Flex 约定的 `yylex()` 接口向 Bison 生成的 parser 提供 token；parser 通过全局根指针 `root_ast` 输出抽象语法树。主程序在解析完成后调用 `SemanticAnalyzer` 的访问接口，对 AST 进行一次完整遍历；语义分析器在内部通过 `SymbolTable` 的 `insert`、`lookup`、`enter_scope`、`exit_scope` 等操作维护符号环境，并通过 `TypeResolver` 查询表达式和 `record` 字段的类型信息。代码生成器同样通过访问者接口遍历 AST，并通过 `generate()` 输出完整的 C 代码字符串。

错误处理方面，各阶段统一通过 `ErrorHandler::instance()` 上报错误，而不是各自直接向控制台打印。这样做的好处是，错误输出格式一致，且主程序可以在适当时机统一检查 `has_errors()` 决定是否继续执行后续阶段。对于需要源码定位的信息，主程序在启动时就把源文件的所有行缓存到 `ErrorHandler` 中，这使得后续诊断输出可以同时展示行号、列号、源码原文和 `^~~~` 形式的定位标记。

3.6 用户接口设计

本项目的用户接口采用命令行方式，形式简洁，便于在本地和平台环境中统一使用。基本调用方式为 `pascc -i input.pas`，系统会自动根据输入文件名生成对应的 `.c` 输出文件；也支持通过 `-o` 参数显式指定输出路径。对于课程平台环境，系统同样可以通过命令行接收输入源文件，并输出同目录或指定位置的目标文件，满足自动化评测脚本的调用需求。

与图形化界面相比，命令行接口更符合编译器类程序的使用习惯，也更利于脚本化测试和平台集成。考虑到课程验收时需要现场展示，命令行方式还能直接配合源码示例、错误样例和生成结果进行演示，便于说明系统的输入、处理和输出全过程。因此，本项目总体设计中没有额外引入 GUI 层，而是把工作重点放在编译主流程、错误诊断和可验证输出上。

4. 详细设计

4.1 词法分析模块详细设计

词法分析模块由 `lexer.l` 描述，并由 Flex 自动生成扫描器。该模块的输入是源程序字符流，输出是供 Bison 使用的 token 序列。实现时采用了“规则匹配 + 动作代码”的基本方式：对关键字、标识符、数字常量、字符与字符串常量、运算符、界符分别编写词法规则，在匹配成功后返回相应 token，并在需要时向语法分析阶段传递语义值。

考虑到 Pascal-S 对关键字大小写不敏感，词法规则在识别保留字和标识符时采取统一化策略，既保证用户输入灵活，又避免后续 parser 和语义阶段因大小写差异造成歧义。对整型、实型、布尔型、字符型和字符串型字面量，扫描器会在返回 token 前完成基础值转换，使 parser 在构建 AST 时可以直接使用对应的字面量节点。与此同时，词法分析阶段还负责过滤空白符和注释，避免无意义字符进入语法归约流程。

本项目在词法设计中特别强化了位置信息维护。扫描器在识别 token 时不仅更新行号，还维护 token 的起始列号和长度信息，这些信息随后被错误处理模块用于生成 `caret diagnostics`。也就是说，词法分析模块不仅承担“分词”职责，还为后续语法错误和语义错误的源码级定位提

供原始坐标数据。该设计使项目的错误提示不再局限于简单行号，而能够进一步输出源码原文及箭头标记。

4.2 语法分析与 AST 构建详细设计

语法分析模块由 `parser.y` 定义，通过 Bison 生成 LALR 语法分析器。该模块的输入是词法分析器提供的 token 流，输出是整个程序的抽象语法树 `ProgramNode`。在实现上，本项目并未把 `parser` 设计成只做“合法/不合法”的判定，而是在各条文法规则的归约动作中同步构造 AST 节点。这使得语法分析完成时，系统已经获得一棵可供后续统一处理的结构化中间表示。

为了支持 Pascal-S 中的程序头、常量与变量声明、函数/过程定义、数组访问、record 字段访问、条件语句、循环语句和函数/过程调用等结构，`parser.y` 中按语言构造分别设计了对应规则，并在动作代码中创建表达式节点、语句节点和声明节点。例如，赋值语句会被组织成 `AssignmentNode`，二元运算被组织成 `BinaryExpressionNode`，函数调用与过程调用分别建模为不同的调用节点，record 字段访问则映射为独立的 `RecordAccessNode`。

较早期的课程实验代码中，`parser` 往往容易混入大量与类型判断、符号表更新有关的副作用逻辑。为了改善这一问题，本项目在当前版本中引入了 `ParserState` 模块，把解析阶段确实需要保存的临时状态集中管理，例如参数列表、局部声明暂存、数组与 record 附加信息、待挂接的子程序声明等。这样做并没有让 `parser` 变成“零状态”，但显著减少了分散的全局变量，使语法分析模块更接近“根据文法构建 AST”的单一职责设计。

4.3 语义分析、符号表与类型系统详细设计

语义分析模块由 `semantic_analyzer.*` 实现，是本项目后期架构调整中最核心的一步。主程序在完成 AST 构建后，创建 `SemanticAnalyzer` 对象并对根节点执行一次完整遍历。该遍历过程一方面负责建立和维护独立的符号表，另一方面负责检查上下文相关的语义约束，从而把“程序结构正确”和“程序含义正确”这两类问题分离开来。

符号表模块 `symbol_table.*` 采用作用域栈式设计。进入程序体、函数体或过程体时，语义分析器进入新作用域；退出相应块时再弹出该作用域。每个 `SymbolEntry` 中保存名字、种类、数据类型、返回类型、数组信息、record 信息、参数列表以及是否为引用参数等属性。通过这种结构，系统可以完成同一作用域内的重定义检查，也可以在表达式分析和赋值检查时按就近原则查找外层声明。

在具体检查内容上，语义分析模块主要覆盖以下几类问题：标识符是否先声明后使用；同一作用域中是否重复声明；赋值语句左值与右值类型是否兼容；条件表达式是否满足布尔约束；函数与过程调用的实参与形参数量、顺序和类型是否匹配；`var` 参数对应的实参是否为可取地址的左值；record 字段访问是否合法。对于 Pascal-S 中“通过给函数名赋值来返回结果”的规则，语义分析器还需要识别当前函数上下文，确保该语义在后续代码生成时能够被正确映射。

为了避免类型推断逻辑在语义阶段和代码生成阶段重复实现，本项目引入了独立的 `TypeResolver` 模块。该模块负责统一解析表达式类型、数组元素类型以及 record 字段类型。`SemanticAnalyzer` 在需要判断表达式是否合法时直接调用这一共享能力，而不是在自身内部再维护一套并行的类型规则。这样既降低了重复代码，也减少了“语义阶段判断通过但代码生成阶段再次猜错类型”的风险。

4.4 代码生成模块详细设计

代码生成模块由 `codegen.*` 实现，其输入为已经通过语义检查的 AST，输出为完整的 C 源程序字符串。生成器总体采用访问者模式实现：对不同节点类型分别定义 `visit` 方法，在遍历 AST

的过程中逐步输出头文件、全局声明、函数/过程定义、语句和表达式。为了保证生成结果在结构上完整，`generate()` 会先完成程序级预处理，再输出 `record` 定义、前向声明、全局对象和主函数体。

Pascal-S 与 C 之间存在多处语义差异，因此代码生成模块的关键难点不是“拼接字符串”，而是完成正确的语义映射。第一，Pascal-S 数组可以具有非零下界，而 C 数组默认从 0 开始，因此在生成数组访问表达式时需要减去定义时记录的下界偏移。第二，Pascal-S 的 `var` 参数本质上是引用传递，而 C 中更自然的映射方式是指针参数，因此代码生成器需要在形参声明、实参传递以及实参是否取址之间保持一致。第三，Pascal-S 中的 `record` 结构需要映射为 C 的 `struct`，并在嵌套 `record` 场景下递归生成字段结构定义和字段访问表达式。

函数返回值的处理也是代码生成中的一个关键点。Pascal-S 函数常通过“对函数名赋值”实现返回，而 C 使用显式 `return`。本项目在生成函数体时为每个非 `void` 子程序引入内部返回值变量 `_retval`，函数体中若出现对当前函数名的赋值，则在 C 代码中改写为对 `_retval` 的赋值，最后在函数尾部统一 `return _retval;`。这一处理方式既保留了 Pascal-S 的原始语义，又让目标代码更接近 C 语言的常规写法。

除了语义映射外，代码生成器还承担一定的工程性职责。例如，它会收集函数的局部变量和参数信息，用于后续声明输出；会在需要时预留临时变量，以处理具有副作用的实参求值；会根据数据类型选择合适的 `printf/scanf` 格式控制串；还会区分语句级调用和表达式级调用，保证生成代码既可编译又符合测试平台要求。这些设计共同构成了当前项目后端实现的主体。

```
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> .\build-mingw2\pascc.exe .\course_tests\success\record_nested_access.pas -o .\docs\presentation\assets\record_nested_access_demo.c
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> Get-Content .\course_tests\success\record_nested_access.pas

program RecordNestedAccess(input, output);
var
  point : record
    x : integer;
    inner : record
      y : integer;
    end;
end;
begin
  point.x := 3;
  point.inner.y := point.x + 4;
  write(point.inner.y);
end.
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> Get-Content .\docs\presentation\assets\record_nested_access_demo.c

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <stdbool.h>

struct pas_record_1 {
  int y;
};

struct pas_record_2 {
  int x;
  struct pas_record_1 inner;
};

struct pas_record_2 point;

int main() {
  point.x = 3;
  point.inner.y = point.x + 4;
  printf("%d", point.inner.y);
  return 0;
}
```

Figure 2: `record` 嵌套字段访问样例及其生成的 C 代码片段

4.5 错误处理与测试支撑详细设计

错误处理模块由 `error.*` 实现，采用单例 `ErrorHandler` 统一管理。各阶段并不直接向控制台零散打印错误，而是通过 `lexical_error`、`syntax_error`、`semantic_error` 和 `codegen_error` 等接口把错误信息、行号、列号和长度上报给错误处理器。主程序在适当阶段查询 `has_errors()`，并在必要时统一调用 `print_errors()` 输出。这样可以保证不同阶段的错误具有一致的格式，同时便于控制何时终止后续编译流程。

在诊断展示上，本项目把错误处理设计成“源码缓存 + 位置信息 + 统一格式”的组合机制。主程序在启动时预先把源文件逐行读入 `ErrorHandler`；词法分析阶段记录 `token` 的行列坐标；语法和语义阶段在发现问题时上报相应位置及长度。最终输出时，系统不仅给出错误类别和文字

说明，还会打印出错源码行，并在对应位置下方绘制 ^~~~ 形式的箭头标记。这种 caret diagnostics 设计比传统“只报 line X error”更利于调试，也更适合课程答辩现场演示。

在语法错误处理策略上，当前系统不再停留在“遇到第一处错误立即终止”的最简单模式，而是结合 Bison 的 error 产生式实现了有限的 panic-mode 恢复。具体做法是在声明列表、语句序列、复合语句结尾以及 repeat-until 等结构周围设置同步点，当缺失分号或出现局部语法片段错误时，parser 可以丢弃当前错误片段并在分号、END、UNTIL 等同步记号处恢复，从而继续报告后续语法问题。这样做一方面避免了同一处错误引发大量重复报错，另一方面也能在一份源文件中暴露多处相互独立的语法问题，更符合课程设计答辩中对“错误处理与恢复策略”的要求。

测试支撑部分则围绕“平台通过 + 本地补充验证”两个目标展开。项目保留了课程平台要求的公开与隐藏测试适配能力，同时在仓库中新增 course_tests/ 与 user_tests/ 两类测试资源。其中 course_tests/ 采用成功样例、错误样例分离的组织方式：成功样例要求编译、生成 C、运行后输出精确匹配；错误样例要求编译失败并且错误输出包含规定的诊断子串。此外，项目还补充了严格的 C11 生成代码检查，用于验证输出 .c 文件在更严格编译选项下仍保持规范性。通过这些测试支撑，系统不仅能证明“功能可用”，还能够证明“结果可复现、输出可验证”。

5. 源程序清单

考虑到本项目源文件数量较多，且课程设计报告正文不适合逐页粘贴完整代码，因此本章采用“核心源文件列表 + 文件职责说明 + 组织方式说明”的形式给出源程序清单。完整源代码随课程设计提交材料一并提供，本章重点说明项目的主要实现文件及其功能划分。

5.1 源程序组织方式

当前项目采用“核心源码目录 + 测试目录 + 提交材料目录”的组织方式。核心编译器实现位于 src/，包括词法分析、语法分析、AST、语义分析、类型系统、代码生成和错误处理等模块；课程自建测试位于 course_tests/ 和 user_tests/；最终验收准备材料位于 submission/，用于整理报告、分组表、验收登记表、使用说明和答辩材料。

从编译器实现角度看，src/ 目录是系统主体，采用头文件与实现文件配对的组织方式。诸如 semantic_analyzer.*、symbol_table.*、type_resolver.*、codegen.* 等文件对应清晰的模块边界，便于在课程答辩时按“词法分析、语法分析、语义分析、代码生成、错误处理”五个层次分别说明。该组织方式也使得后续功能扩展和问题定位更加直接。

5.2 核心源文件清单

文件	类别	主要职责
main.cpp	主控入口	解析命令行参数，组织 parse -> semantic -> codegen 主流程，负责输入输出文件处理和错误终止控制
lexer.l	词法分析	定义 Flex 词法规则，识别关键字、标识符、字面量、注释及位置信息
parser.y	语法分析	定义 Bison 文法规则，在归约动作中构建 AST

parser_state.h/.cpp	解析状态	集中保存 parser 需要的临时状态，减少分散的全局可变数据
ast.h/.cpp	抽象语法树	定义 AST 节点体系与访问者接口，作为中间表示供语义分析和代码生成共享
token.h	公共定义	定义 token、数据类型及部分公共枚举/辅助结构
semantic_analyzer.h/.cpp	语义分析	遍历 AST，完成标识符绑定、作用域管理、类型检查和语义合法性验证
symbol_table.h/.cpp	符号表	维护作用域栈、符号项插入与查找，支撑语义分析阶段的上下文信息管理
type_resolver.h/.cpp	类型系统	统一完成表达式类型推断、数组元素类型解析和 record 字段类型查询
codegen.h/.cpp	代码生成	将语义正确的 AST 翻译为等价 C 程序，并处理数组偏移、record、引用参数与函数返回值等映射
error.h/.cpp	错误处理	统一收集和输出词法、语法、语义及代码生成阶段的错误，支持 caret diagnostics

5.3 测试与辅助文件清单

除核心源码外，项目还包含测试、构建和验收支撑文件。主要内容如下。

路径/文件	类别	主要职责
course_tests/success/	成功样例	存放课程自建的成功编译与正确输出测试用例
course_tests/errors/	错误样例	存放课程自建的语法/语义错误测试及其期望错误子串
course_tests/run_course_tests.ps1	测试脚本	自动运行课程自建测试并比较输出结果
course_tests/run_strict_c_checks.ps1	严格检查	对生成的 C 代码执行严格 C11 编译验证
user_tests/	回归样例	存放开发阶段用于快速验证功能的本地测试程序
submission/	提交材料	组织最终报告、分组表、验收登记表、使用说明和答辩材料草稿
scripts/	辅助脚本	存放平台与调试相关的辅助脚本和历史测试工具

5.4 代码组织与实现特点

从源程序组织方式可以看出，本项目并不是把所有逻辑堆叠在单一文件中，而是围绕编译器流水线进行分层组织。词法和语法部分分别依赖 Flex 与 Bison 自动生成底层分析器，但分析规则与归约动作仍由开发者在 lexer.l 和 parser.y 中明确控制；语义与类型系统则由独立模块承担，不再混杂在 parser 中；代码生成模块建立在 AST 与共享类型信息之上，负责把 Pascal-S 的语义映射为标准 C 输出。

在命名与实现风格方面，项目主要采用“模块名 + 功能职责”的方式组织文件，例如 `semantic_analyzer`、`symbol_table`、`type_resolver`、`codegen` 等名称均能直接反映模块用途。头文件主要负责类、结构体和接口声明，实现文件负责算法与细节逻辑，便于阅读时先把握总体接口，再深入实现细节。对课程设计而言，这种组织方式既有助于多人分工说明，也便于在答辩现场快速定位相关源码。

5.5 源程序提交说明

最终提交时，完整源程序将以仓库或压缩包形式一并提供，保证验收教师能够直接查看 `src/`、`course_tests/`、`user_tests/` 和相关构建文件。考虑到课程设计报告正文篇幅有限，本章所列清单已经覆盖了最主要的实现文件和辅助材料；若需进一步查看具体实现，可直接结合仓库目录和代码注释进行阅读。该组织方式兼顾了报告简洁性和源码可追溯性，也符合课程设计提交“源程序 + 可运行程序 + 测试用例 + 使用说明”的整体要求。

6. 程序测试

本章测试内容基于当前验收准备阶段的实际结果撰写，重点说明测试环境、测试分类、代表性用例及结果分析。后续如需补充更多运行截图、平台页面截图或附录样例，可在不改变本章整体结构的前提下继续扩展。

6.1 测试环境

本项目的本地测试主要在 Windows 环境下完成，核心工具链包括 Flex、Bison、GCC / MinGW、PowerShell 和 CMake。编译器主体由 C++ 实现，输入为 Pascal-S 源文件，输出为等价的 C 文件，再通过 GCC 对生成结果做编译和运行验证。对于课程平台部分，则按照头歌平台要求，在指定环境下将编译器生成的可执行文件放置到规定目录，并由平台公开与隐藏测试自动完成样例驱动验证。

在本地测试流程中，通常先调用 `pascc` 将 `.pas` 翻译为 `.c`，再使用 GCC 编译并运行目标程序，对控制台输出与预期输出文件进行比较。对于错误样例，则不再进入目标代码编译阶段，而是直接检查编译器输出的错误信息是否包含预期关键字和定位信息。除此之外，本项目还单独增加了一组严格 C11 检查，用于验证生成的 C 代码在更严格编译选项下仍具有规范性。

6.2 测试内容与测试分类

结合课程要求和本项目扩展特性，测试内容主要分为四类。

第一类是平台兼容性测试。该部分面向头歌平台的公开测试与隐藏测试，重点验证编译器是否满足课程评测环境要求，以及在较大覆盖范围内是否能够稳定完成 Pascal-S 到 C 的翻译。

第二类是课程自建成功样例测试。该部分对应 `course_tests/success` 中的样例，重点覆盖平台基础用例之外、但在本项目中具有代表性的功能点，例如 `record` 与嵌套字段访问、数组下界偏移、函数作为语句调用、`var` 参数配合数组元素、布尔/整数 `not`、程序头中的 `input`、`output` 以及多参数 `write` 等。

第三类是课程自建错误样例测试。该部分对应 `course_tests/errors` 中的样例，用于验证语义错误与语法错误的诊断能力，包括未声明标识符、`record` 字段不存在、引用参数要求左值、赋值类型不匹配以及缺失分号等情况。

第四类是生成代码规范性测试。该部分通过 `run_strict_c_checks.ps1` 对所有成功样例生成的 C 代码执行严格编译，编译参数包括 `-std=c11 -pedantic-errors -Wall -Wextra -Werror`，用于验证生成结果不仅“能跑”，而且在更严格的 C 语言检查下仍保持可接受的规范性。

6.3 代表性测试用例

为了避免在正文中机械罗列全部样例，本节选取几类最具代表性的测试进行说明。

(1) record 与多级字段访问测试。样例 `record_nested_access.pas` 定义了带嵌套 record 的变量，并执行 `point.inner.y := point.x + 4` 这类多级字段访问与赋值。其测试目的在于同时验证 parser、语义分析与代码生成三层对 record 的支持是否一致，预期结果是编译成功并输出正确数值。该样例既是扩展功能演示，也是较典型的加分项验证。

(2) 数组下界偏移测试。样例 `array_lower_bound_shift.pas` 检查 Pascal-S 数组从非零下界开始定义时，生成的 C 代码是否正确完成下标平移。其测试目的在于验证目标代码语义映射是否正确，避免出现“程序能编译但访问位置错误”的隐蔽问题。

(3) 引用参数与左值约束测试。成功样例 `var_parameter_array_element.pas` 用于验证数组元素作为 var 参数时能够被正确翻译；错误样例 `semantic_reference_requires_lvalue.pas` 则用于验证当实参不是左值时，语义分析是否会给出明确报错。两者结合可以同时证明系统对“合法引用传递”和“非法引用传递”的区分能力。

(4) 错误诊断测试。样例 `semantic_record_unknown_field.pas` 用于验证 record 字段不存在时能否输出带源码原文与箭头定位的语义错误；样例 `syntax_missing_semicolon.pas` 用于验证语法错误能否在缺失分号等典型场景下给出明确定位。这部分测试不仅检查“能否报错”，也检查“报错是否便于理解和展示”。

(5) 语句扩展测试。样例 `repeat_until_basic.pas` 用于验证 `repeat ... until` 在多语句循环体场景下能否被正确翻译；样例 `semantic_repeat_until_condition_type.pas` 则用于验证当 until 条件不是布尔兼容表达式时，语义分析能否给出明确报错。该组样例用于证明新扩展并非只停留在词法或语法层面，而是已经贯通到语义分析和代码生成。

(6) 平台总体回归测试。平台公开与隐藏测试覆盖了基本语法结构、声明、表达式、控制流、函数过程、数组、输入输出等广泛场景。本项目最终在头歌平台达到 95/95 全通过，说明系统在课程规定范围内已经具备较稳定的正确性。

6.4 测试结果

当前阶段的主要测试结果如下。

测试类别	结果	说明
头歌平台测试	95 / 95	公开测试与隐藏测试全部通过
课程自建成功样例	9 / 9	覆盖 record、repeat-until、数组下界偏移、函数返回、引用参数等功能
课程自建错误样例	7 / 7	覆盖语法错误、语义错误及多错误恢复场景
严格 C11 检查	9 / 9	生成 C 代码通过 <code>-std=c11 -pedantic-errors</code> 编译
用户自建回归样例	8 / 8	覆盖词法、表达式、控制流、函数、数组、record 与 repeat-until 等基础能力

从上述结果可以看出，平台结果证明了系统在课程给定评测范围内的整体正确性；课程自建测试则对扩展功能、错误处理和多错误恢复进行了更有针对性的补充；严格 C11 检查进一步说明生成结果不仅能在宽松条件下运行，也能在较严格的标准约束下通过编译。

6.5 预期结果与实际结果对比

在成功样例中，预期结果通常包括三个层面：第一，Pascal-S 源程序能够被编译器成功翻译为 .c 文件；第二，生成的 .c 文件能够通过 GCC 编译；第三，程序运行后的控制台输出与预期输出文件完全一致。本项目在 `course_tests/success` 和 `user_tests` 中所选的代表样例均满足上述预期，尤其是 `record`、多参数 `write`、数组下界偏移和函数返回值等较容易出错的场景，均已得到正确结果。

在错误样例中，预期结果是不生成错误的目标程序，并输出包含指定关键字和位置信息的诊断文字。当前测试结果表明，未声明标识符、`record` 字段不存在、引用参数不是左值、赋值类型不匹配和缺失分号等场景均能被准确拦截，并给出相应的语法或语义错误说明。对于带 `caret diagnostics` 的样例，输出还能够显示源码原文与箭头定位，这一点对答辩展示也具有直接帮助。

```
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> .\build-mingw2\pascc.exe .\course_tests\errors\semantic_rec
ord_unknown_field.pas
[Semantic Error] line 7, col 9: Unknown record field: missing
      node.missing := 1;
      ^
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler>
```

Figure 3: 语义错误示例：`record` 字段不存在时的 `caret diagnostics` 输出

6.6 测试结果分析

综合平台结果、本地成功样例、本地错误样例和严格 C11 检查，可以认为本项目当前版本已经形成了较完整的测试闭环。平台 95/95 全通过证明基本功能和主要边界情况已经达到课程评测要求；课程自建测试则弥补了平台样例对扩展功能展示不足的问题，使 `record`、错误诊断、自定义验证脚本等内容能够在报告和答辩中被单独说明；严格 C11 检查则提升了对目标代码规范性的把关力度。

当然，测试工作并不意味着系统已经“完全没有改进空间”。从工程角度看，后续仍可继续补充更细粒度的极端样例，例如更复杂的嵌套调用、更多样的 `record` 与数组组合场景以及更丰富的错误恢复案例。但就课程设计验收阶段而言，当前测试体系已经能够较充分地说明：本项目不仅在平台要求下可用，而且在扩展功能、错误提示和目标代码规范性方面也具备较好的可验证性。

7. 课程设计总结

7.1 主要完成工作

经过本次课程设计，本组最终完成了一个面向 Pascal-S 子集的源到源编译器原型系统。该系统能够把 Pascal-S 源程序翻译为等价的 C 程序，并完成从词法分析、语法分析与 AST 构建，到语义分析、错误诊断和目标代码生成的完整流水线。与课程基础要求相比，项目不仅支持常量、变量、函数与过程、表达式、控制流、数组、输入输出等基本功能，还进一步扩展实现了 `record` 结构与多级字段访问、带源码定位的 `caret diagnostics`、自建课程测试集以及严格 C11 生成代码检查。

从结果上看，项目在头歌平台最终达到 95/95 全通过，本地也建立了覆盖成功样例、错误样例和生成代码规范性的测试体系。这说明本次课程设计并非停留在“少量样例可运行”的层面，而是形成了一个具有较完整编译流程、较清晰模块边界和较明确验证支撑的工程原型。

7.2 设计与实现过程中的主要问题

在系统开发过程中，最突出的困难主要集中在三个方面。

第一，语法分析、语义分析与代码生成之间的职责边界在早期版本中并不清晰。最初的实现更接近“在 parser 中一边归约一边处理其他逻辑”的方式，虽然在小样例上能工作，但随着函数、数组、record 和错误处理能力逐步增加，代码可维护性明显下降。为了解决这一问题，项目后期将语义分析从 parser 中拆分出来，构建了独立的 SemanticAnalyzer、SymbolTable 与 TypeResolver 模块，使 parser 回归到“按文法建树”的主要职责，整体架构也因此更加清晰。

第二，Pascal-S 与 C 之间存在多处语义差异，导致代码生成不是简单的语法替换。例如数组下界偏移、var 参数引用传递、函数名赋值式返回值以及 record 到 struct 的映射，都需要在后端做专门处理。这部分问题的解决过程使本组认识到：代码生成的难点并不在于“输出字符串”，而在于是否准确地把源语言语义映射到目标语言机制之上。

第三，错误处理和测试支撑在课程设计中具有比预期更高的重要性。仅依赖平台通过率，难以充分说明系统对错误输入、边界情况和扩展功能的处理能力。为此，本项目在后期补充了统一错误处理器、源码级 caret diagnostics、自建成功/错误样例以及严格生成代码检查，使系统从“功能正确性”进一步走向“可验证性”和“可展示性”。

7.3 分工协作与个人收获

本项目采用“按模块划分、组长统一集成”的方式组织工作，围绕词法分析、语法分析、AST、语义分析、类型系统、目标代码验证和测试支撑等环节展开分工。组长负责总体架构设计、主程序组织、模块联调和最终验收整合；其余成员分别围绕词法分析、语法分析、AST 结构、语义分析与符号表、类型系统与错误诊断、目标代码验证与测试支撑等模块展开准备和说明。通过这种方式，系统既保留了编译器分层实现的结构特征，也便于答辩时按照模块逐一说明设计思路。

从学习收获来看，本次课程设计使成员对编译原理中的多个核心概念形成了更具体的理解。词法分析不再只是课堂中的正则表达式与自动机，而是落实为可运行的 token 扫描器；语法分析不再只是文法推导，而是通过 Bison 规则真正构造出 AST；语义分析则让“作用域”“符号表”“类型检查”等抽象概念变成了可验证的程序逻辑；代码生成阶段则进一步说明了源语言语义与目标语言机制之间的转换关系。整体而言，本次课程设计帮助我们课堂知识与实际工程实现建立了更紧密的联系。

7.4 不足与后续改进方向

尽管本项目已经完成课程要求并取得较好的测试结果，但从工程角度看仍存在进一步提升空间。首先，部分模块虽然已经具备较清晰边界，但代码规模仍有继续优化和细化的余地，例如代码生成模块仍可以继续拆分为更小的职责单元，以进一步提升可读性。其次，当前错误处理已经支持较好的 caret diagnostics，但在更复杂的错误恢复场景、连续错误抑制策略和诊断信息丰富度方面仍可增强。再次，测试体系已经形成基本闭环，但仍可以继续补充更极端的边界样例，以及更加自动化的报告生成与统计流程。

如果后续继续迭代，本项目可优先考虑三个方向：其一，进一步增强模块文档与注释，使初学者能够更快理解各阶段职责；其二，补充更丰富的语言扩展能力，例如更复杂的类型组合和更完整的运行时支持；其三，完善提交材料与测试输出之间的联动，使课程设计报告、使用说明和自动化测试结果能够更方便地统一维护。

7.5 总结

总体而言，本次 Pascal-S 编译器课程设计达到了预期目标。项目不仅完成了课程要求的基本翻译功能，还在语义分析架构、`record` 扩展、错误诊断和测试体系等方面做出了较为完整的实现和整理。通过这一过程，本组对编译器从前端分析到后端映射的整体结构有了更系统的认识，也更加理解了“理论正确”“工程可维护”“结果可验证”三者之间的关系。该项目最终形成的代码、测试和文档材料，也为后续验收答辩提供了较为扎实的支撑。